

LAPORAN PRAKTIKUM PEKAN 8

STRUKTUR DATA

Disusun Oleh:

Syasya Halwa Gazwani

(2511531018)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Alvero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
DASAR TEORI	2
2.1 ShellSort_2511531018	2
2.1.1 Kode Program	2
2.1.2 Langkah Kerja	3
2.1.3 Output	5
2.2 QuickSort_2511531018	5
2.2.1. Kode Program	5
2.2.2. Langkah Kerja	6
2.2.3 Output	10
2.3 MergeSort_2511531018.....	10
2.3.1 Kode Program	10
2.3.2 Langkah Kerja	11
2.4.3 Output	14
2.4 BubbleSortGUI_2511531018.....	15
2.4.1 Kode Program	15
2.4.2 Langkah Kerja	19
2.4.3 Output	22

2.5	MergeSortGUI_2511531018.....	24
2.5.1	Kode Program	24
2.5.2	Langkah Kerja	29
2.5.3	Output	32
2.6	TUGAS.....	36
2.6.1	Lagu_2511531018	36
2.6.2	Sorting_2511531018	37
2.6.3	Output	38
2.6.4	Penjelasan	38
BAB III	PENUTUP	40
DAFTAR PUSTAKA.....		41

BAB I PENDAHULUAN

1.1 Latar Belakang

Pengolahan merupakan hal terpenting dalam pemrograman agar informasi dapat disusun dengan rapi dan mudah digunakan. Salah satu pengolahan data yang digunakan dalam pemrograman yaitu algoritma sorting yang berguna untuk mengurutkan data-data yang tersedia. Pengurutan data membantu pengguna dalam mencari, membaca, dan mengelola informasi dengan lebih cepat dan teratur. Pada praktikum ini digunakan algoritma Shell Sort, Quick Sort, Bubble Sort, dan Merge Sort untuk proses pengurutan data.

1.2 Tujuan

- 1.2.1 Dapat memahami konsep dasar algoritma sorting pada struktur data.
- 1.2.2 Mengurutkan data-data dengan menggunakan algoritma Shell Sort, Quick Sort, Bubble Sort, dan Merge Sort.
- 1.2.3 Melatih kemampuan dalam mengelola data menggunakan ArrayList dan object class.

1.3 Manfaat Praktikum

- 1.3.1 Menambah pemahaman tentang penggunaan ArrayList, object class, dan method sorting.
- 1.3.2 Membuat proses belajar struktur data menjadi lebih interaktif dan menarik melalui visualisasi sorting.
- 1.2.4 Membantu pengguna memahami perbedaan cara kerja Shell Sort, Quick Sort, Bubble Sort, dan Merge Sort.

BAB II PEMBAHASAN

DASAR TEORI

Algoritma pengurutan (*sorting*) yang digunakan untuk menyusun data secara terstruktur. Meskipun memiliki tujuan yang sama, setiap algoritma memiliki cara kerja dan tingkat efisiensi yang berbeda. BubbleSort mengurutkan data dengan cara membandingkan dua elemen yang bersebelahan secara berulang dan menukarnya jika posisinya salah. Proses ini diulang terus-menerus sampai tidak ada lagi pertukaran. ShellSort pengembangan dari metode Insertion Sort. Shell Sort membagi data yang berjauhan menjadi sub-kelompok yang lebih kecil, mengurutkannya, lalu secara bertahap mengurangi jarak antar elemen tersebut. MergeSort membagi daftar data menjadi dua bagian sama besar secara berulang sampai tiap bagian hanya berisi satu elemen. Kemudian, potongan-potongan tersebut digabungkan kembali (*merge*) sambil diurutkan dari yang terkecil ke terbesar (atau sebaliknya). QuickSort memilih satu elemen acak sebagai patokan (*pivot*). Elemen-elemen lain kemudian diolah menjadi dua bagian yang nilainya lebih kecil dari pivot ditempatkan di sisi kiri, dan yang lebih besar di sisi kanan

2.1 ShellSort_2511531018

2.1.1 Kode Program

```
package pekan8_2511531018;

public class Shellsort_2511531018 {
    public static void shellSort_2511531018(int[] A) {
        int n_1018 = A.length;
        int gap_1018 = n_1018 / 2;
        while (gap_1018 > 0) {
            for (int i_1018 = gap_1018; i_1018 < n_1018;
i_1018++) {
                int temp_1018 = A[i_1018];
                int j_1018 = i_1018;
                while (j_1018 >= gap_1018 && A[j_1018 -
gap_1018] > temp_1018) {
                    A[j_1018] = A[j_1018 -
gap_1018];
                    j_1018 = j_1018 - gap_1018;
                }
                A[j_1018] = temp_1018;
            }
            gap_1018 = gap_1018 / 2;
        }
    }
}
```

```

    }
    gap_1018 = gap_1018 / 2;
}

public static void main(String[] args) {
    int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

    System.out.print("Sebelum: ");
    printArray(data);

    shellSort_2511531018(data);

    System.out.print("Sesudah (Shell Sort): ");
    printArray(data);
}

public static void printArray(int[] arr) {
    for (int i_1018 : arr) System.out.print(i_1018 + "
");
    System.out.println();
}

```

Gambar 2.1.1 Kode program ShellSort_2511531018.

2.1.2 Langkah Kerja

2.1.2.1 Menggunakan method `shellSort_2511531018` untuk melakukan proses pengurutan data, lalu menyimpan jumlah elemen yang ada pada array ke dalam integer `n`, menentukan jarak antar elemen dengan nilai awal adalah setengah dari panjang array dengan menggunakan integer `gap`, dan melakukan perulangan selama nilai `gap` lebih besar dari 0.

```

public static void shellSort_2511531018(int[] A) {
    int n_1018 = A.length;
    int gap_1018 = n_1018 / 2;
    while (gap_1018 > 0) {

```

Gambar 2.1.2.1 Method `shellSort_2511531018`.

2.1.2.2 Melakukan perulangan untuk menelusuri elemen array, nilai elemen yang di proses disimpan sementara dalam integer `temp`, `int j` digunakan sebagai indeks pada proses pergeseran elemen, dan perulangan `while` membandingkan elemen dengan elemen lain yang berjarak sesuai nilai `gap`.

```

for (int i_1018 = gap_1018; i_1018 < n_1018; i_1018++)
{
    int temp_1018 = A[i_1018];

```

```

        int j_1018 = i_1018;
        while (j_1018 >= gap_1018 &&
A[j_1018 - gap_1018] > temp_1018) {

```

Gambar 2.1.2.2 Perulangan.

- 2.1.2.3 Elemen yang lebih besar digeser pada posisi berikutnya, setelah posisi yang tepat ditemukan, nilai yang disimpan pada int temp ditempatkan pada posisi tersebut.

```

A[j_1018] = A[j_1018 - gap_1018];
        j_1018 = j_1018 -
gap_1018;
    }
    A[j_1018] = temp_1018;
}
gap_1018 = gap_1018 / 2;
}

```

Gambar 2.1.2.3 Pemindahan elemen.

- 2.1.2.4 .

Method main yaitu tempat program dijalankan dengan membuat array yang berisi elemen-elemen yang belum terurut dan menampilkan isi array yang belum terurut.

```

public static void main(String[] args) {
    int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

    System.out.print("Sebelum: ");
    printArray(data);
}

```

Gambar 2.1.2.4 Method main.

- 2.1.2.5 Memanggil method shellSort_2511531018 untuk mengurutkan elemen-elemen dan menampilkan array yang telah terurut.

```

shellSort_2511531018(data);

    System.out.print("Sesudah (Shell Sort): ");
    printArray(data);
}

```

Gambar 2.1.2.5 Method main.

- 2.1.2.6 Menggunakan method printArray untuk menampilkan seluruh isi array ke layar, melakukan perulangan untuk mengambil setiap elemen dalam array, dan menampilkan seluruh elemen.

```

public static void printArray(int[] arr) {
    for (int i_1018 : arr)
System.out.print(i_1018 + " ");
}

```

```
System.out.println();
```

Gambar 2.1.2.6 Method main.

2.1.3 Output

```
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

Gambar 2.1.3 Output ShellSort_2511531018.

2.2 QuickSort_2511531018

2.2.1. Kode Program

```
package pekan8_2511531018;

public class Quicksort_2511531018 {
    static void swap(int[] arr_1018, int i_1018, int j_1018) {
        int temp_1018 = arr_1018[i_1018];
        arr_1018[i_1018] = arr_1018[j_1018];
        arr_1018[j_1018] = temp_1018;
    }
    // metode tambahan untuk mengatur pivot menggunakan Median-
    of-Three
    static void medianOfThree_2511531018(int[] arr_1018, int
    low_1018, int high_1018) {
        int mid_1018 = low_1018 + (high_1018 - low_1018) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_1018[low_1018] > arr_1018[mid_1018]) {
            swap(arr_1018, low_1018, mid_1018);
        }
        if (arr_1018[low_1018] > arr_1018[high_1018]) {
            swap(arr_1018, low_1018, high_1018);
        }
        if (arr_1018[mid_1018] > arr_1018[high_1018]) {
            swap(arr_1018, mid_1018, high_1018);
        }
        swap(arr_1018, mid_1018, high_1018);
    }
    static int partition_2511531018(int[] arr_1018, int
    low_1018, int high_1018) {
        // panggil fungsi medianofThree sebelum menentukan
        pivot
        medianOfThree_2511531018(arr_1018, low_1018,
        high_1018);

        int pivot_1018=arr_1018[high_1018]; // sekarang
        arr[high] sudah berisi nilai median
        int i_1018 = (low_1018 - 1);

        for (int j_1018 = low_1018; j_1018 <= high_1018 - 1;
        j_1018++) {
```

```

        // jika elemen saat ini lebih kecil dari atau
sama engan pivot
        if (arr_1018[j_1018] < pivot_1018) {
            // increment indeks elemen yang lebih
kecil
            i_1018++;
            swap(arr_1018, i_1018, j_1018);
        }
        swap(arr_1018, i_1018 + 1, high_1018);
        return (i_1018 + 1);
    }

    static void quickSort_2511531018(int[] arr_1018, int
low_1018, int high_1018) {
        if (low_1018 < high_1018) {
            int pi_1018 = partition_2511531018(arr_1018,
low_1018, high_1018);
            quickSort_2511531018(arr_1018, low_1018,
pi_1018 - 1);
            quickSort_2511531018(arr_1018, pi_1018 + 1,
high_1018);
        }
    }

    static void printArr_2511531018(int[] arr_1018) {
        for (int i_1018 = 0; i_1018 < arr_1018.length ;
i_1018++) {
            System.out.print(arr_1018[i_1018] + " ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[] arr_1018 = { 10, 7, 8, 9, 1, 5 };
        int N_1018=arr_1018.length;
        System.out.print("Data sebelum diurutkan: ");
        printArr_2511531018(arr_1018);

        quickSort_2511531018(arr_1018, 0, N_1018 - 1);

        System.out.print("Data terurut quicksort: ");
        printArr_2511531018(arr_1018);
    }
}

```

Gambar 2.2.1 Kode program QuickSort_2511531018.

2.2.2. Langkah Kerja

2.2.2.1 Menggunakan algoritma QuickSort_2511531018 dengan teknik Median-of-Three untuk pemilihan pivot agar proses pengurutan berjalan sesuai, menukar posisi dua elemen dalam array, lalu menyimpan nilai sementara, nilai pada indeks j

dipindahkan ke indeks i, dan nilai yang disimpan di variabel sementara ditempatkan pada indeks j.

```
public class Quicksort_2511531018 {
    static void swap(int[] arr_1018, int i_1018, int
j_1018) {
        int temp_1018 = arr_1018[i_1018];
        arr_1018[i_1018] = arr_1018[j_1018];
        arr_1018[j_1018] = temp_1018;
    }
}
```

Gambar 2.2.2.1 Method QuickSort_2511531018.

- 2.2.2.2 Method medianOfThree digunakan untuk menentukan pivot dengan memilih nilai tengah dari 3 elemen. Menentukan indeks tengah dari bagian array yang sedang di proses.

```
static void medianOfThree_2511531018(int[] arr_1018,
int low_1018, int high_1018) {
    int mid_1018 = low_1018 + (high_1018 -
low_1018) / 2;
}
```

Gambar 2.2.2.2 Median-Of-Three.

- 2.2.2.3 Membandingkan elemen awal dan elemen tengah, lalu menukar posisi elemen awal dan tengah jika elemen awal lebih besar. Setelah itu membandingkan elemen awal dan akhir dan menukarnya jika elemen awal lebih besar dari elemen akhir. Setelah itu, membandingkan elemen tengah dengan elemen akhir dan menukarnya jika diperlukan. Nilai median dipindahkan ke posisi akhir array dan digunakan sebagai pivot.

```
if (arr_1018[low_1018] > arr_1018[mid_1018]) {
    swap(arr_1018, low_1018, mid_1018);
}
if (arr_1018[low_1018] > arr_1018[high_1018])
{
    swap(arr_1018, low_1018, high_1018);
}
if (arr_1018[mid_1018] > arr_1018[high_1018])
{
    swap(arr_1018, mid_1018, high_1018);
}
swap(arr_1018, mid_1018, high_1018);
}
```

Gambar 2.2.2.3 Menukar posisi elemen.

- 2.2.2.4 Method ini digunakan untuk membagi array menjadi dua bagian berdasarkan pivot.

```
static int partition_2511531018(int[] arr_1018, int low_1018, int high_1018) {
```

Gambar 2.2.2.4 Method partition_2511531018.

- 2.2.2.5 Memanggil method Median-of-Three untuk menentukan pivot, menentukan pivot dari elemen terakhir yang sudah berisi nilai median, menyimpan indeks elemen yang lebih kecil dari pivot, dan perulangan untuk memeriksa setiap elemen dalam array.

```
medianOfThree_2511531018(arr_1018, low_1018, high_1018);

    int pivot_1018=arr_1018[high_1018]; //
    sekarang arr[high] sudah berisi nilai median
    int i_1018 = (low_1018 - 1);

    for (int j_1018 = low_1018; j_1018 <=
    high_1018 - 1; j_1018++) {
```

Gambar 2.2.2.5 Method Median-Of-Three.

- 2.2.2.6 Menempatkan elemen yang lebih kecil pada sisi kiri pivot, menambah indeks elemen yang lebih kecil dan menukarkan posisi elemen yang lebih kecil pada sisi kiri pivot. Setelah semua elemen diperiksa, pivot ditempatkan pada posisi yang benar, lalu program mengembalikan indeks posisi pivot.

```
if (arr_1018[j_1018] < pivot_1018) {
    // increment indeks elemen yang
    lebih kecil
    i_1018++;
    swap(arr_1018, i_1018, j_1018);
}
}
swap(arr_1018, i_1018 + 1, high_1018);
return (i_1018 + 1);
}
```

Gambar 2.2.2.6 Memindahkan elemen.

- 2.2.2.7 Menggunakan method quickSort untuk mengurutkan elemen, dilanjutkan dengan memastikan bagian array masih memiliki satu elemen. Memanggil method partition untuk mendapatkan posisi pivot, lalu mengurutkan bagian kiri pivot dan kanan pivot.

```
static void quickSort_2511531018(int[] arr_1018, int low_1018, int high_1018) {
```

```

        if (low_1018 < high_1018) {
            int pi_1018 =
partition_2511531018(arr_1018, low_1018, high_1018);
            quickSort_2511531018(arr_1018,
low_1018, pi_1018 - 1);
            quickSort_2511531018(arr_1018, pi_1018
+ 1, high_1018);
        }

```

Gambar 2.2.2.7 Method QuickSort.

- 2.2.2.8 Menggunakan method printArr untuk menampilkan seluruh isi array. Melakukan perulangan untuk membaca setiap elemen array, lalu menampilkan elemen array.

```

static void printArr_2511531018(int[] arr_1018) {
    for (int i_1018 = 0; i_1018 < arr_1018.length
; i_1018++) {
        System.out.print(arr_1018[i_1018] + "
");
    }
    System.out.println();
}

```

Gambar 2.2.2.8 Method PrintArr.

- 2.2.2.9 Dilanjutkan dengan method main sebagai tempat berjalannya program dengan membuat array yang berisi elemen-elemen yang akan diurutkan, menyimpan jumlah elemen array tersebut, dan menampilkan sata sebelum melakukan pengurutan.

```

public static void main(String[] args) {
    int[] arr_1018 = { 10, 7, 8, 9, 1, 5 };
    int N_1018=arr_1018.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_2511531018(arr_1018);
}

```

Gambar 2.2.2.9 Method main.

- 2.2.2.10 Menggunakan algoritma quickSort untuk mengurutkan data dan menampilkan data setelah diurutkan.

```

quickSort_2511531018(arr_1018, 0, N_1018 - 1);

    System.out.print("Data terurut quicksort: ");
    printArr_2511531018(arr_1018);
}

```

Gambar 2.2.2.10 Algoritma quickSort.

2.2.3 Output

```
Data sebelum diurutkan: 10 7 8 9 1 5
Data terurut quicksort: 1 5 7 8 9 10
```

Gambar 2.2.3 Output QuickSort_2511531018.

2.3 MergeSort_2511531018

2.3.1 Kode Program

```
package pekan8_2511531018;

public class MergeSort_2511531018 {
    void merge_2511531018(int[] arr_1018, int l_1018, int
m_1018, int r_1018) {
        // Find sizes of two subarrays to be merged
        int n1_1018 = m_1018 - l_1018 + 1;
        int n2_1018 = r_1018 - m_1018;
        /* Create temp arrays */
        int L_1018[] = new int[n1_1018];
        int R_1018[] = new int[n2_1018];
        /* Copy data to temp arrays */
        for (int i_1018 = 0; i_1018 < n1_1018; ++i_1018)
            L_1018[i_1018] = arr_1018[l_1018 + i_1018];
        for (int j_1018 = 0; j_1018 < n2_1018; ++j_1018)
            R_1018[j_1018] = arr_1018[m_1018 + 1 +
j_1018];
        int i_1018 = 0, j_1018 = 0;
        // Initial index of merged subarray array
        int k_1018 = l_1018;
        while (i_1018 < n1_1018 && j_1018 < n2_1018) {
            if (L_1018[i_1018] <= R_1018[j_1018]) {
                arr_1018[k_1018] = L_1018[i_1018];
                i_1018++;
            } else {
                arr_1018[k_1018] = R_1018[j_1018];
                j_1018++;
            }
            k_1018++;
        }
        /* Copy remaining elements of L[] if any */
        while (i_1018 < n1_1018) {
            arr_1018[k_1018] = L_1018[i_1018];
            i_1018++;
            k_1018++;
        }
        /* Copy remaining elements of R[] if any */
        while (j_1018 < n2_1018) {
            arr_1018[k_1018] = R_1018[j_1018];
            j_1018++;
            k_1018++;
        }
    }
}
```

```

void sort_2511531018(int arr_1018[], int l_1018, int
r_1018) {
    if (l_1018 < r_1018) {
        // Find the middle point
        int m_1018 = (l_1018 + r_1018) / 2;
        // Sort first and second halves
        sort_2511531018(arr_1018, l_1018, m_1018);
        sort_2511531018(arr_1018, m_1018 + 1,
r_1018);

        // Merge the sorted halves
        merge_2511531018(arr_1018, l_1018, m_1018,
r_1018);
    }
}
/* A utility function to print array of size n */
static void printArray_2511531018(int arr_1018[]) {
    int n_1018 = arr_1018.length;
    for (int i_1018 = 0; i_1018 < n_1018; ++i_1018)
        System.out.print(arr_1018[i_1018] + " ");
    System.out.println();
}
public static void main(String[] args) {
    int arr_1018[] = { 12, 11, 13, 5, 6, 7 };
    System.out.println("Sebelum terurut");
    printArray_2511531018(arr_1018);
    MergeSort_2511531018 ob = new
MergeSort_2511531018();
    ob.sort_2511531018(arr_1018, 0, arr_1018.length -
1);

    System.out.println("\nSesudah Terurut menggunakan
merge Sort");
    printArray_2511531018(arr_1018);
}
}
}

```

Gambar 2.3.1 Kode program MergeSort_2511531018.

2.3.2 Langkah Kerja

2.3.2.1 Menggunakan method mergeSort untuk menggabungkan dua subarray yang sudah terurut menjadi satu array yang terurut.

```

void merge_2511531018(int[] arr_1018, int l_1018, int
m_1018, int r_1018) {

```

Gambar 2.3.2.1 Method MergeSort.

2.3.2.2 Menghitung jumlah elemen paada subarray kiri dan subarray kanan.

```

int n1_1018 = m_1018 - l_1018 + 1;
int n2_1018 = r_1018 - m_1018;

```

Gambar 2.3.2.2 Method MergeSort.

- 2.3.2.3 Membuat array sementara untuk menyimpan data bagian kiri dan bagian kanan.

```
int L_1018[] = new int[n1_1018];
int R_1018[] = new int[n2_1018];
```

Gambar 2.3.2.3 Method MergeSort..

- 2.3.2.4 Menyalin data dari array utama ke array sementara bagian kiri, lalu menyalin data dari array utama ke array sementara bagian kanan, dan menelusuri array kiri dan kanan.

```
for (int i_1018 = 0; i_1018 < n1_1018; ++i_1018)
    L_1018[i_1018] = arr_1018[l_1018 +
i_1018];
for (int j_1018 = 0; j_1018 < n2_1018;
++j_1018)
    R_1018[j_1018] = arr_1018[m_1018 + 1 +
j_1018];
int i_1018 = 0, j_1018 = 0;
```

Gambar 2.3.2.4 Method MergeSort.

- 2.3.2.5 Menentukan posisi awal pada array utama yang akan diisi kembali dan membandingkan elemen pada array kiri dan kanan selama keduanya masih memiliki data. Jika elemen pada array kiri lebih kecil atau sama dengan elemen pada array kanan, maka elemen kiri dimasukkan ke array utama. Menyalin elemen kiri ke array utama dan berpindah ke elemen berikutnya. Jika elemen kanan lebih kecil, maka elemen kanan dimasukkan ke array utama.

```
int k_1018 = l_1018;
while (i_1018 < n1_1018 && j_1018 < n2_1018)
{
    if (L_1018[i_1018] <= R_1018[j_1018]) {
        arr_1018[k_1018] =
L_1018[i_1018];
        i_1018++;
    } else {
        arr_1018[k_1018] =
R_1018[j_1018];
        j_1018++;
    }
    k_1018++;
}
```

Gambar 2.3.2.5 Perpindahan elemen.

- 2.3.2.6 Jika masih ada elemen yang tersisa pada array kiri, maka elemen tersebut disalin ke array utama, lalu menyalin seluruh sisa elemen kiri satu per satu.

```
while (i_1018 < n1_1018) {
    arr_1018[k_1018] = L_1018[i_1018];
    i_1018++;
    k_1018++;
}
```

gambar 2.3.2.6 Menyalin elemen.

- 2.3.2.7 Jika masih ada elemen yang tersisa pada array kanan, maka elemen tersebut disalin ke array utama, lalu menyalin seluruh sisa elemen kiri satu per satu.

```
while (j_1018 < n2_1018) {
    arr_1018[k_1018] = R_1018[j_1018];
    j_1018++;
    k_1018++;
}
```

Gambar 2.3.2.7 Menyalin elemen.

- 2.3.2.8 Method sort digunakan untuk membagi array menjadi bagian-bagian yang lebih kecil dengan memastikan bahwa bagian array masih memiliki lebih dari satu elemen.

```
void sort_2511531018(int arr_1018[], int l_1018, int r_1018) {
    if (l_1018 < r_1018) {
```

Gambar 2.3.2.8 Method sort.

- 2.3.2.9 Menentukan indeks tengah array, mengurutkan bagian kiri array dan kanan array, menggabungkan kembali kedua bagian yang sudah terurut.

```
// Find the middle point
int m_1018 = (l_1018 + r_1018) / 2;
// Sort first and second halves
sort_2511531018(arr_1018, l_1018, m_1018);
sort_2511531018(arr_1018, m_1018 + 1, r_1018);
// Merge the sorted halves
merge_2511531018(arr_1018, l_1018, m_1018, r_1018);
}
```

Gambar 2.3.2.9 mengurutkan elemen.

- 2.3.2.10 Method printArray untuk menampilkan seluruh isi array dengan menyimpan jumlah elemen array, melakukan

perulangan untuk membaca seluruh elemen array, dan menampilkan setiap elemen array.

```
static void printArray_2511531018(int arr_1018[]) {
    int n_1018 = arr_1018.length;
    for (int i_1018 = 0; i_1018 < n_1018;
    ++i_1018)
        System.out.print(arr_1018[i_1018] + "
");
    System.out.println();
}
```

Gambar 2.3.2.10 Method PrintArray.

2.3.2.11 Membuat array yang berisi data yang belum terurut, menampilkan isi array sebelum dilakukan pengurutan.

```
public static void main(String[] args) {
    int arr_1018[] = { 12, 11, 13, 5, 6, 7 };
    System.out.println("Sebelum terurut");
    printArray_2511531018(arr_1018);
}
```

Gambar 2.3.2.11 Method main.

2.3.2.12 Membuat objek dari class MergeSort_2511531018. Memanggil method sort untuk mengurutkan seluruh elemen array menggunakan Merge Sort dan menampilkan hasil array setelah proses pengurutan selesai.

```
MergeSort_2511531018 ob = new MergeSort_2511531018();
ob.sort_2511531018(arr_1018, 0,
arr_1018.length - 1);
System.out.println("\nSesudah Terurut
menggunakan merge Sort");
printArray_2511531018(arr_1018);
```

Gambar 2.3.2.6 Memanggil Method sort.

2.4.3 Output

```
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
```

Gambar 2.3.3 Output MergeSort_2511531018.

2.4 BubbleSortGUI_2511531018

2.4.1 Kode Program

```
package pekan8_2511531018;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class BubbleSortGUI_2511531018 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1018;
    private int[] array_1018;
    private JLabel[] labelArray_1018;
    private JButton stepButton_1018;
    private JButton resetButton_1018;
    private JButton setButton_1018;
    private JTextField inputField_1018;
    private JPanel panelArray_1018;
    private JTextArea stepArea_1018;

    private int i_1018 = 1, j_1018;
    private boolean sorting_1018 = false;
    private int stepCount_1018 = 1;

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511531018() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);
        contentPane_1018 = new JPanel();
        contentPane_1018.setBorder(new EmptyBorder(5, 5, 5,
5));

        setContentPane(contentPane_1018);
        contentPane_1018.setLayout(null);
        setTitle("Bubble sort langkah per langkah");
        setSize(750, 400);
        setLocationRelativeTo(null);
    }
}
```

```

        setLayout(new BorderLayout());

        // panel input
        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField_1018 = new JTextField(30);
        setButton_1018 = new JButton("Set Array");
        inputPanel.add(new JLabel("Masukkan angka (pisahkan
dengan koma): "));
        inputPanel.add(inputField_1018);
        inputPanel.add(setButton_1018);

        // panel array visual
        panelArray_1018 = new JPanel();
        panelArray_1018.setLayout(new FlowLayout());

        // panel kontrol
        JPanel controlPanel = new JPanel();
        stepButton_1018 = new JButton("Langkah
Selanjutnya");
        resetButton_1018 = new JButton("Reset");
        stepButton_1018.setEnabled(false);
        controlPanel.add(stepButton_1018);
        controlPanel.add(resetButton_1018);

        // area teks untuk log langkah-langkah
        stepArea_1018 = new JTextArea(8,60);
        stepArea_1018.setEditable(false);
        stepArea_1018.setFont(new Font("Monospaced",
Font.PLAIN, 14));
        JScrollPane scrollPane = new
JScrollPane(stepArea_1018);

        // tambahkan ke panel ke frame
        add(inputPanel, BorderLayout.NORTH);
        add(panelArray_1018, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        // event set array
        setButton_1018.addActionListener(e ->
setArrayFromInput_1018());

        // event langkah selanjutnya
        stepButton_1018.addActionListener(e ->
performStep_1018());

        // event reset
        resetButton_1018.addActionListener(e ->
reset_1018());
    }
    private void setArrayFromInput_1018() {
        String text_1018 = inputField_1018.getText().trim();
        if (text_1018.isEmpty()) return;
        String[] parts_1018 = text_1018.split(",");
        array_1018 = new int[parts_1018.length];
    }

```

```

        try {
            for (int k_1018 = 0; k_1018 < parts_1018.length;
k_1018++) {
                array_1018[k_1018] =
Integer.parseInt(parts_1018[k_1018].trim());
            }
        } catch (NumberFormatException e_1018) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka " + "yang dipisahkan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        i_1018 = 0;
        j_1018 = 0;
        stepCount_1018 = 1;
        sorting_1018 = true;

        stepButton_1018.setEnabled(true);
        stepArea_1018.setText("");
        panelArray_1018.removeAll();

        labelArray_1018 = new JLabel[array_1018.length];
        for (int k_1018 = 0; k_1018 < array_1018.length;
k_1018++) {
            labelArray_1018[k_1018] = new
JLabel(String.valueOf(array_1018[k_1018]));
            labelArray_1018[k_1018].setFont( new Font("Arial",
Font.BOLD, 24));
            labelArray_1018[k_1018].setOpaque(true);
            labelArray_1018[k_1018].setBackground(Color.WHITE);

            labelArray_1018[k_1018].setBorder(BorderFactory.createLineBor
der(Color.BLACK));
            labelArray_1018[k_1018].setPreferredSize(new
Dimension(50, 50));

            labelArray_1018[k_1018].setHorizontalAlignment(SwingConstants
.CENTER);
            panelArray_1018.add(labelArray_1018[k_1018]);
        }

        panelArray_1018.revalidate();
        panelArray_1018.repaint();
    }

    private void performStep_1018() {
        if (!sorting_1018 || i_1018 >= array_1018.length - 1) {
            sorting_1018 = false;
            stepButton_1018.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
            return;
        }

        resetHighlights_1018();
        StringBuilder stepLog_1018 = new StringBuilder();

```

```

        labelArray_1018[j_1018].setBackground(Color.CYAN);
        labelArray_1018[j_1018 + 1].setBackground(Color.CYAN);

        if (array_1018[j_1018] > array_1018[j_1018 + 1]) {
            // Swap
            int temp_1018 = array_1018[j_1018];
            array_1018[j_1018] = array_1018[j_1018 + 1];
            array_1018[j_1018 + 1] = temp_1018;
            labelArray_1018[j_1018].setBackground(Color.RED);
            labelArray_1018[j_1018 +
1].setBackground(Color.RED);

            stepLog_1018.append("Langkah
").append(stepCount_1018).append(": Menukar elemen ke-")
                .append(j_1018).append("
").append(array_1018[j_1018 + 1]).append(" dengan ke-")
                .append(j_1018 + 1).append("
").append(array_1018[j_1018]).append(")\n");
        } else {
            stepLog_1018.append("Langkah
").append(stepCount_1018).append(": Tidak ada pertukaran
antara ke-")
                .append(j_1018).append(" dan ke-
").append(j_1018 + 1).append(")\n");
        }

        stepLog_1018.append("Hasil:
").append(arrayToString_1018(array_1018)).append("\n\n");
        stepArea_1018.append(stepLog_1018.toString());
        updateLabels_1018();
        j_1018++;

        if (j_1018 >= array_1018.length - i_1018 - 1) {
            j_1018 = 0;
            i_1018++;
        }

        stepCount_1018++;

        if (i_1018 >= array_1018.length - 1) {
            sorting_1018 = false;
            stepButton_1018.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        }
    }

    private void updateLabels_1018() {
        for (int k_1018 = 0; k_1018 < array_1018.length;
k_1018++) {

            labelArray_1018[k_1018].setText(String.valueOf(array_1018[k_1
018]));
        }
    }

    private void resetHighlights_1018() {

```

```

        for (JLabel label_1018 : labelArray_1018) {
            label_1018.setBackground(Color.WHITE);
        }
    }

    private void reset_1018() {
        inputField_1018.setText("");
        panelArray_1018.removeAll();
        panelArray_1018.revalidate();
        panelArray_1018.repaint();
        stepArea_1018.setText("");
        stepButton_1018.setEnabled(false);
        sorting_1018 = false;
        i_1018 = 0;
        j_1018 = 0;
        stepCount_1018 = 1;
    }

    private String arrayToString_1018(int[] arr_1018) {
        StringBuilder sb_1018 = new StringBuilder();
        for (int k_1018 = 0; k_1018 < arr_1018.length;
k_1018++) {
            sb_1018.append(arr_1018[k_1018]);
            if (k_1018 < arr_1018.length - 1) {
                sb_1018.append(", ");
            }
        }
        return sb_1018.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubbleSortGUI_2511531018 gui = new
BubbleSortGUI_2511531018();
            gui.setVisible(true);
        });
    }
}

```

Gambar 2.4.1 Kode program BubbleSortGUI_2511531018.

2.4.2 Langkah Kerja

2.4.2.1 Method ini dijalankan ketika tombol Set Array ditekan, memisahkan angka berdasarkan tanda koma dan membuat array sesuai jumlah data yang dimasukkan. Mengubah input berupa teks menjadi tipe data integer.

```

private void setArrayFromInput_1018() {
    String text_1018 =
inputField_1018.getText().trim();
    if (text_1018.isEmpty()) return;
    String[] parts_1018 = text_1018.split(",");
    array_1018 = new int[parts_1018.length];
}

```

```
try {
    for (int k_1018 = 0; k_1018 <
parts_1018.length; k_1018++) {
        array_1018[k_1018] =
Integer.parseInt(parts_1018[k_1018].trim());
```

Gambar 2.4.2.1 Set array from input..

- 2.4.2.2** Membuat label yang digunakan untuk menampilkan setiap elemen array secara visual. Method ini juga berfungsi untuk menginisialisasi ulang variabel sorting sebelum proses pengurutan dimulai.

```
labelArray_1018 = new JLabel[array_1018.length];
for (int k_1018 = 0; k_1018 < array_1018.length;
k_1018++) {
```

Gambar 2.4.2.2 Set array from input.

- 2.4.2.3** Method ini merupakan bagian utama algoritma Bubble Sort yang dijalankan setiap kali tombol Langkah Selanjutnya ditekan, membandingkan dua elemen yang berdekatan, dan menukar posisi kedua elemen jika elemen kiri lebih besar daripada elemen kanan.

```
private void performStep_1018() {
    if (array_1018[j_1018] > array_1018[j_1018 + 1]) {
        // Swap
        int temp_1018 = array_1018[j_1018];
        array_1018[j_1018] = array_1018[j_1018 + 1];
        array_1018[j_1018 + 1] = temp_1018;
```

Gambar 2.4.2.3 PerformStep.

- 2.4.2.4** Method ini digunakan untuk memperbarui isi label setelah terjadi perubahan data pada array dan menampilkan nilai terbaru array pada tampilan GUI.

```
private void updateLabels_1018() {
    for (int k_1018 = 0; k_1018 < array_1018.length;
k_1018++) {

labelArray_1018[k_1018].setText(String.valueOf(array_1
018[k_1018]));
```

Gambar 2.4.2.4 UpdateLabels.

- 2.4.2.5** Method ini digunakan untuk mengembalikan warna seluruh label menjadi putih dan warna direset agar penanda langkah sebelumnya tidak ikut tampil pada langkah berikutnya.

```
private void resetHighlights_1018() {
```

```
for (JLabel label_1018 : labelArray_1018) {
    label_1018.setBackground(Color.WHITE);
}
```

Gambar 2.4.2.5 ResetHighlights.

- 2.4.2.6** Method ini dijalankan saat tombol Reset ditekan, menghapus input dan log langkah pengurutan, dan menonaktifkan tombol langkah sampai pengguna memasukkan array baru.

```
private void reset_1018() {
    inputField_1018.setText("");
    stepArea_1018.setText("");
    stepButton_1018.setEnabled(false);
}
```

Gambar 2.4.2.6 Reset.

- 2.4.2.7** Method ini digunakan untuk mengubah isi array menjadi string dan menambahkan setiap elemen array ke dalam objek StringBuilder.

```
private String arrayToString_1018(int[] arr_1018) {
    StringBuilder sb_1018 = new StringBuilder();
    for (int k_1018 = 0; k_1018 < arr_1018.length;
k_1018++) {
        sb_1018.append(arr_1018[k_1018]);
        if (k_1018 < arr_1018.length - 1) {
            sb_1018.append(", ");
        }
    }
}
```

Gambar 2.4.2.7 ArrayToString.

- 2.4.2.8** Membuat objek GUI dan menampilkan jendela aplikasi sehingga pengguna dapat mulai menggunakan program.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        BubbleSortGUI_2511531018 gui =
new BubbleSortGUI_2511531018();
        gui.setVisible(true);
    });
}
```

Gambar 2.4.2.8 Method main.

2.4.3 Output

Bubble sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

3	5	2	1
---	---	---	---

Bubble sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

3	5	2	1
---	---	---	---

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 3, 5, 2, 1

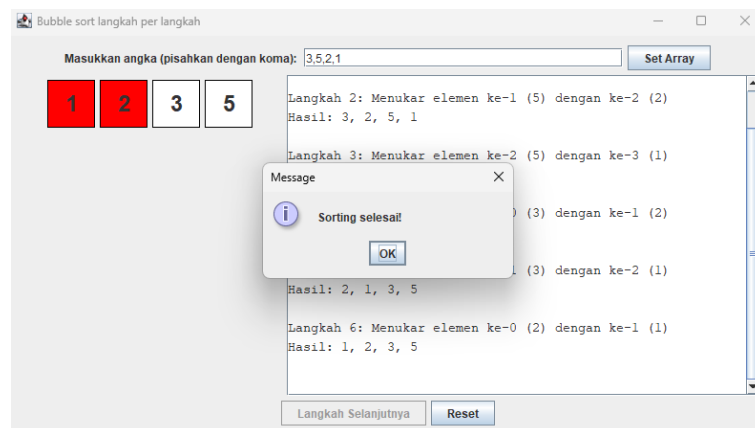
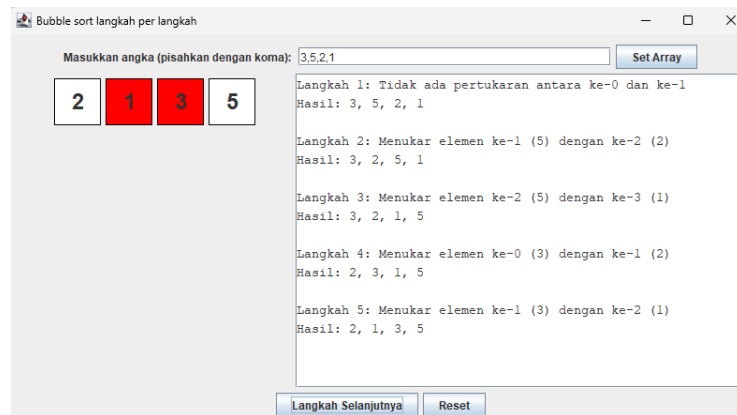
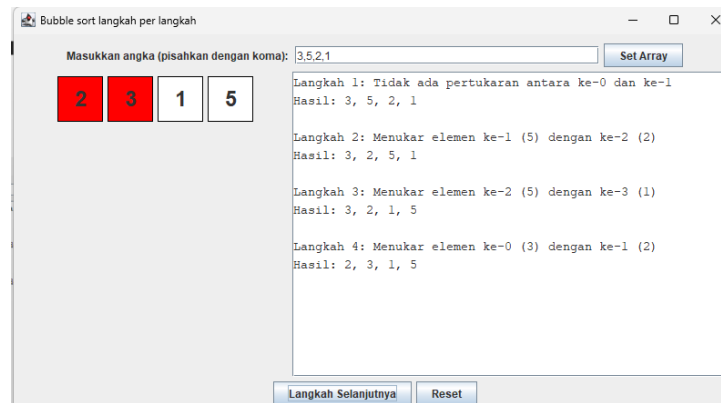
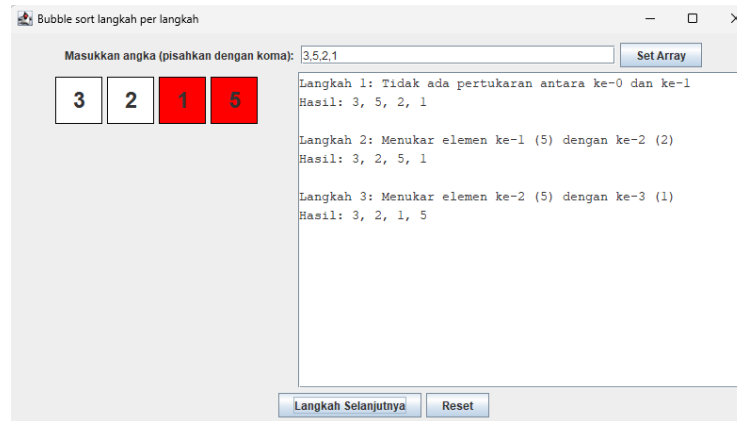
Bubble sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

3	2	5	1
---	---	---	---

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 3, 5, 2, 1

Langkah 2: Menukar elemen ke-1 (5) dengan ke-2 (2)
Hasil: 3, 2, 5, 1



Gambar 2.4.3 Output BubbleSort_2511531018.

2.5 MergeSortGUI_2511531018

2.5.1 Kode Program

```
package pekan8_2511531018;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;
import java.util.LinkedList;
import java.util.Queue;

public class MergeSortGUI_2511531018 extends JFrame{
    private static final long serialVersionUID = 1L;
    private JPanel contentPane_1018;
    private int[] array_1018;
    private JLabel[] labelArray_1018;
    JButton stepButton_1018;
    private JButton resetButton_1018;
    JButton setButton_1018;
    private JTextField inputField_1018;
    private JPanel panelArray_1018;
    private JTextArea stepArea_1018;

    private int i_1018 = 1, j_1018;
    private boolean sorting_1018 = false;
    private int stepCount_1018 = 1;
    private Queue<int[]> mergeQueue_1018 = new
LinkedList<>();

    private boolean isMerging_1018 = false;
    private boolean copying_1018 = false;
    private int left_1018;
    private int mid_1018;
```

```

        private int right_1018;
        private int k_1018;
        private int[] temp_1018;

        /**
         * Create the frame.
         * @return
         */
        public MergeSortGUI_2511531018() {

            setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            setBounds(100, 100, 450, 300);
            contentPane_1018 = new JPanel();
            contentPane_1018.setBorder(new EmptyBorder(5,
5, 5, 5));

            setContentPane(contentPane_1018);
            contentPane_1018.setLayout(null);
            setTitle("Merge sort langkah per langkah");
            setSize(750, 400);
            setLocationRelativeTo(null);
            setLayout(new BorderLayout());

            // panel input
            JPanel inputPanel = new JPanel(new
FlowLayout());
            inputField_1018 = new JTextField(30);
            setButton_1018 = new JButton("Set Array");
            inputPanel.add(new JLabel("Masukkan angka
(pisahkan dengan koma): "));
            inputPanel.add(inputField_1018);
            inputPanel.add(setButton_1018);

            // panel array visual
            panelArray_1018 = new JPanel();
            panelArray_1018.setLayout(new FlowLayout());

            // panel kontrol
            JPanel controlPanel = new JPanel();
            stepButton_1018 = new JButton("Langkah
Selanjutnya");
            resetButton_1018 = new JButton("Reset");
            stepButton_1018.setEnabled(false);
            controlPanel.add(stepButton_1018);
            controlPanel.add(resetButton_1018);

            // area teks untuk log langkah-langkah
            stepArea_1018 = new JTextArea(8,60);
            stepArea_1018.setEditable(false);
            stepArea_1018.setFont(new Font("Monospaced",
Font.PLAIN, 14));
            JScrollPane scrollPane = new
JScrollPane(stepArea_1018);

            // tambahkan ke panel ke frame
            add(inputPanel, BorderLayout.NORTH);
            add(panelArray_1018, BorderLayout.CENTER);
            add(controlPanel, BorderLayout.SOUTH);

```

```

        add(scrollPane, BorderLayout.EAST);

        // event set array
        setButton_1018.addActionListener(e ->
setArrayFromInput_1018());

        // event langkah selanjutnya
        stepButton_1018.addActionListener(e ->
performStep_1018());

        // event reset
        resetButton_1018.addActionListener(e ->
reset_1018());
    }

    private void setArrayFromInput_1018() {
        String text_1018 =
inputField_1018.getText().trim();
        if (text_1018.isEmpty()) return;
        String[] parts_1018 = text_1018.split(",");
        array_1018 = new int[parts_1018.length];

        try {
            for (int i_1018 = 0; i_1018 <
parts_1018.length; i_1018++) {
                array_1018[i_1018] =
Integer.parseInt(parts_1018[i_1018].trim());
            }
        } catch (NumberFormatException e_1018) {
            JOptionPane.showMessageDialog(this,
"Masukkan hanya angka!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }

        labelArray_1018 = new JLabel[array_1018.length];
        panelArray_1018.removeAll();

        for (int i_1018 = 0; i_1018 < array_1018.length;
i_1018++) {
            labelArray_1018[i_1018] = new
JLabel(String.valueOf(array_1018[i_1018]));
            labelArray_1018[i_1018].setFont(new
Font("Arial", Font.BOLD, 24));
            labelArray_1018[i_1018].setOpaque(true);

            labelArray_1018[i_1018].setBackground(Color.WHITE);

            labelArray_1018[i_1018].setBorder(BorderFactory.createLineBorder(
Color.BLACK));
            labelArray_1018[i_1018].setPreferredSize(new
Dimension(50, 50));

            labelArray_1018[i_1018].setHorizontalAlignment(SwingConstants
.CENTER);

```

```

panelArray_1018.add(labelArray_1018[i_1018]);

        mergeQueue_1018.clear();
        generateMergeSteps_1018(0, array_1018.length
- 1);

        stepButton_1018.setEnabled(true);
        stepArea_1018.setText("");

        stepCount_1018 = 1;
        isMerging_1018 = false;

        panelArray_1018.revalidate();
        panelArray_1018.repaint();
    }
}

private void generateMergeSteps_1018(int
left_1018, int right_1018) {
    if (left_1018 >= right_1018) {
        return;
    }
    int mid_1018 = (left_1018 + right_1018) / 2;
    generateMergeSteps_1018(left_1018,
mid_1018);
    generateMergeSteps_1018(mid_1018 + 1,
right_1018);
    mergeQueue_1018.offer(new int[] {
        left_1018, mid_1018, right_1018
    });
}

private void performStep_1018() {
    resetHighlights_1018();
    if (!isMerging_1018 &&
!mergeQueue_1018.isEmpty()) {
        int[] range_1018 = mergeQueue_1018.poll();
        left_1018 = range_1018[0];
        mid_1018 = range_1018[1];
        right_1018 = range_1018[2];
        temp_1018 = new int[right_1018 - left_1018 +
1];

        i_1018 = left_1018;
        j_1018 = mid_1018 + 1;
        k_1018 = 0;
        copying_1018 = false;
        isMerging_1018 = true;
        stepArea_1018.append("Langkah " +
stepCount_1018++ + ": Mulai merge dari " + left_1018 + " ke "
+ right_1018 + "\n");

        return;
    }

    if (isMerging_1018 && !copying_1018) {
        if (i_1018 <= mid_1018 && j_1018 <=
right_1018) {

```

```

labelArray_1018[i_1018].setBackground(Color.CYAN);
labelArray_1018[j_1018].setBackground(Color.CYAN);
    if (array_1018[i_1018] <=
array_1018[j_1018]) {
        temp_1018[k_1018++] =
array_1018[i_1018++];
    } else {
        temp_1018[k_1018++] =
array_1018[j_1018++];
    }
    stepArea_1018.append("Langkah " +
stepCount_1018++ + ": Bandingkan dan salin elemen\n");
    return;
    } else if (i_1018 <= mid_1018) {
        temp_1018[k_1018++] =
array_1018[i_1018++];
        stepArea_1018.append("Langkah " +
stepCount_1018++ + ": Salin sisa kiri\n");
        return;
    } else if (j_1018 <= right_1018) {
        temp_1018[k_1018++] =
array_1018[j_1018++];
        stepArea_1018.append("Langkah " +
stepCount_1018++ + ": Salin sisa kanan\n");
        return;
    } else {
        copying_1018 = true;
        k_1018 = 0;
        return;
    }
}

    if (copying_1018 && k_1018 < temp_1018.length) {
        array_1018[left_1018 + k_1018] =
temp_1018[k_1018];
        labelArray_1018[left_1018 +
k_1018].setText(String.valueOf(temp_1018[k_1018]));
        labelArray_1018[left_1018 +
k_1018].setBackground(Color.GREEN);
        k_1018++;
        stepArea_1018.append("Langkah " +
stepCount_1018++ + ": Tempelkan ke array utama\n");
        return;
    }

    if (copying_1018 && k_1018 == temp_1018.length)
{
        isMerging_1018 = false;
        copying_1018 = false;
    }

    if (mergeQueue_1018.isEmpty() &&
!isMerging_1018) {
        stepArea_1018.append("Selesai.\n");
        stepButton_1018.setEnabled(false);
    }

```

```

JOptionPane.showMessageDialog(this, "Merge
Sort selesai!");
    }
}

private void resetHighlights_1018() {
    if (labelArray_1018 == null) return;
    for (JLabel label_1018 : labelArray_1018) {
        label_1018.setBackground(Color.WHITE);
    }
}

private void reset_1018() {
    inputField_1018.setText("");
    panelArray_1018.removeAll();
    panelArray_1018.revalidate();
    panelArray_1018.repaint();
    stepArea_1018.setText("");
    stepButton_1018.setEnabled(false);
    mergeQueue_1018.clear();
    isMerging_1018 = false;
    stepCount_1018 = 1;
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        MergeSortGUI_2511531018 gui =
new MergeSortGUI_2511531018();
        gui.setVisible(true);
    });
}
}

```

Gambar 2.6.1 Kode Program MergeSortGUI_2511531018.

2.5.2 Langkah Kerja

2.5.2.1 Method ini dijalankan ketika tombol Set Array ditekan, digunakan untuk memisahkan input berdasarkan tanda koma dan membuat array. Membuat label untuk menampilkan setiap elemen array secara visual dan mengosongkan queue kemudian membuat urutan langkah Merge Sort yang akan digunakan selama proses visualisasi.

```

private void setArrayFromInput_1018() {
    String[] parts_1018 =
text_1018.split(",");
    array_1018 = new int[parts_1018.length];

    try {

```

```

        for (int i_1018 = 0; i_1018 <
parts_1018.length; i_1018++) {
            array_1018[i_1018] =
Integer.parseInt(parts_1018[i_1018].trim());
        }

        labelArray_1018 = new
JLabel[array_1018.length];

        mergeQueue_1018.clear();
        generateMergeSteps_1018(0,
array_1018.length - 1);

```

Gambar 2.5.2.1 setArrayFromInput.

- 2.5.2.2 Method ini digunakan untuk membuat urutan proses merge menggunakan konsep rekursif pada Merge Sort dengan menentukan titik tengah array, membagi array menjadi bagian kiri dan kanan, dan menyimpan informasi proses merge ke dalam queue agar dapat dijalankan langkah demi langkah.

```

private void generateMergeSteps_1018(int
left_1018, int right_1018) {
    int mid_1018 = (left_1018 +
right_1018) / 2;
    generateMergeSteps_1018(left_1018,
mid_1018);
    generateMergeSteps_1018(mid_1018 + 1,
right_1018);
    mergeQueue_1018.offer(new int[] {
        left_1018, mid_1018,
right_1018
    });
}

```

Gambar 2.5.2.2 generateMergeStep.

- 2.5.2.3 Method ini merupakan inti program karena menjalankan proses Merge Sort secara bertahap setiap kali tombol Langkah Selanjutnya ditekan. Mengambil proses merge berikutnya dari queue dan membuat array sementara untuk menyimpan hasil penggabungan. Membandingkan elemen bagian kiri dan kanan, jika elemen kiri lebih kecil, maka elemen tersebut disalin ke array sementara, jika elemen kanan lebih kecil, maka elemen kanan yang disalin. Menyalin sisa elemen pada bagian kiri. Menyalin kembali hasil merge ke array utama dengan memperbarui tampilan visual array.

```

private void performStep_1018() {

```

```

        int[] range_1018 =
mergeQueue_1018.poll();

        temp_1018 = new int[right_1018 -
left_1018 + 1];

        if (array_1018[i_1018] <=
array_1018[j_1018]) {
            temp_1018[k_1018++] =
array_1018[i_1018++];
        } else {
            temp_1018[k_1018++] =
array_1018[j_1018++];
        }

        array_1018[left_1018 + k_1018] =
temp_1018[k_1018];
        labelArray_1018[left_1018 +
k_1018].setText(String.valueOf(temp_1018[k_1018]));
        labelArray_1018[left_1018 +
k_1018].setBackground(Color.GREEN);
        k_1018++;

```

Gambar 2.5.2.3 performStep.

- 2.5.2.4 Method ini digunakan untuk mengembalikan warna seluruh label menjadi putih sebelum langkah berikutnya dijalankan dan menghapus warna penanda pada langkah sebelumnya.

```

private void resetHighlights_1018() {
    if (labelArray_1018 == null) return;
    for (JLabel label_1018 :
labelArray_1018) {
        label_1018.setBackground(Color.WHITE);
    }
}

```

Gambar 2.5.2.4 resetHighlights.

- 2.5.2.5 Method ini dijalankan saat tombol Reset ditekan, lalu menghapus input pengguna, menghapus visualisasi array, menghapus log langkah-langkah MergeSort, dan menghapus seluruh proses merge yang tersimpan dalam queue.

```

private void reset_1018() {
    inputField_1018.setText("");
    panelArray_1018.removeAll();
    panelArray_1018.revalidate();
    panelArray_1018.repaint();
    stepArea_1018.setText("");
    stepButton_1018.setEnabled(false);
    mergeQueue_1018.clear();
    isMerging_1018 = false;
    stepCount_1018 = 1;
}

```

Gambar 2.5.2.5 reset.

2.5.2.6 Method utama yang menjalankan program dengan membuat objek GUI dan menampilkan jendela aplikasi pada pengguna.

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
    {
        MergeSortGUI_2511531018
gui = new MergeSortGUI_2511531018();
gui.setVisible(true);
    });
}
```

Gambar 2.5.2.6 Method main.

2.5.3 Output



Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

5	2	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

2	2	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma):

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3
 Langkah 7: Bandingkan dan salin elemen

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma): 5,2,4,6

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3
 Langkah 7: Bandingkan dan salin elemen
 Langkah 8: Salin sisa kanan

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma): 5,2,4,6

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3
 Langkah 7: Bandingkan dan salin elemen
 Langkah 8: Salin sisa kanan
 Langkah 9: Tempelkan ke array utama

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma): 5,2,4,6

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3
 Langkah 7: Bandingkan dan salin elemen
 Langkah 8: Salin sisa kanan
 Langkah 9: Tempelkan ke array utama
 Langkah 10: Tempelkan ke array utama

Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma): 5,2,4,6

2	5	4	6
---	---	---	---

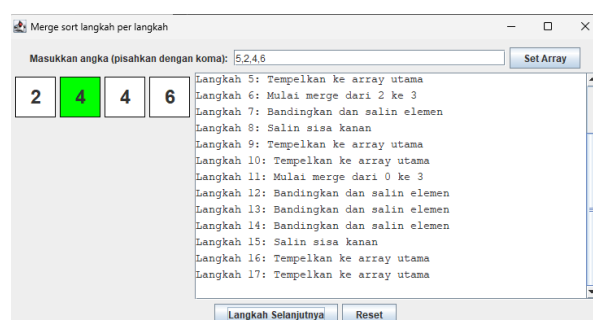
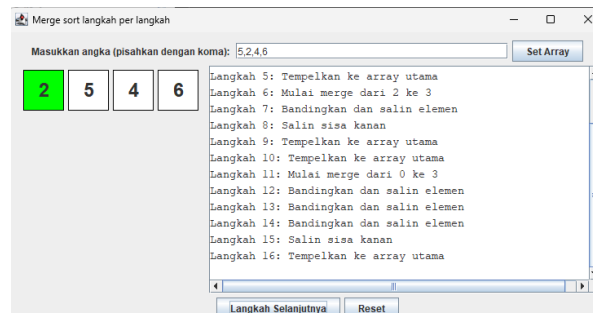
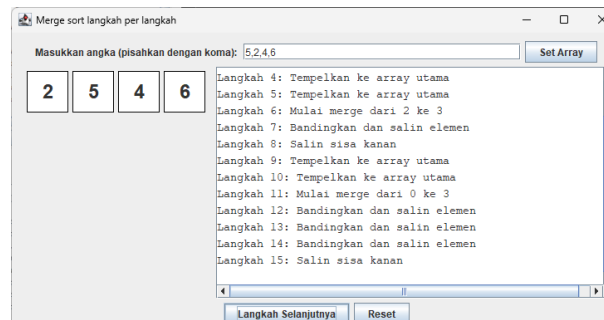
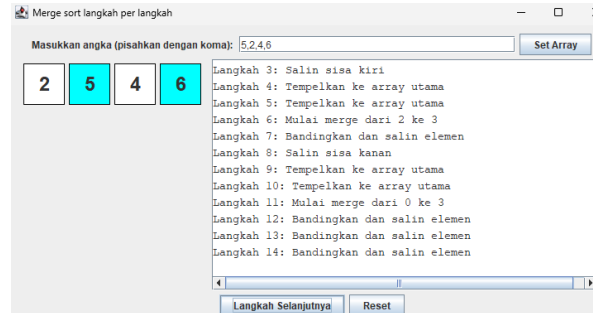
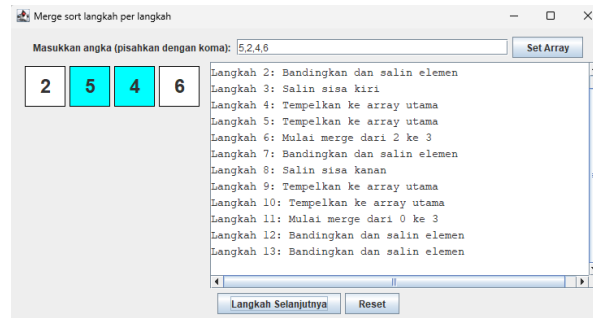
Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3
 Langkah 7: Bandingkan dan salin elemen
 Langkah 8: Salin sisa kanan
 Langkah 9: Tempelkan ke array utama
 Langkah 10: Tempelkan ke array utama
 Langkah 11: Mulai merge dari 0 ke 3

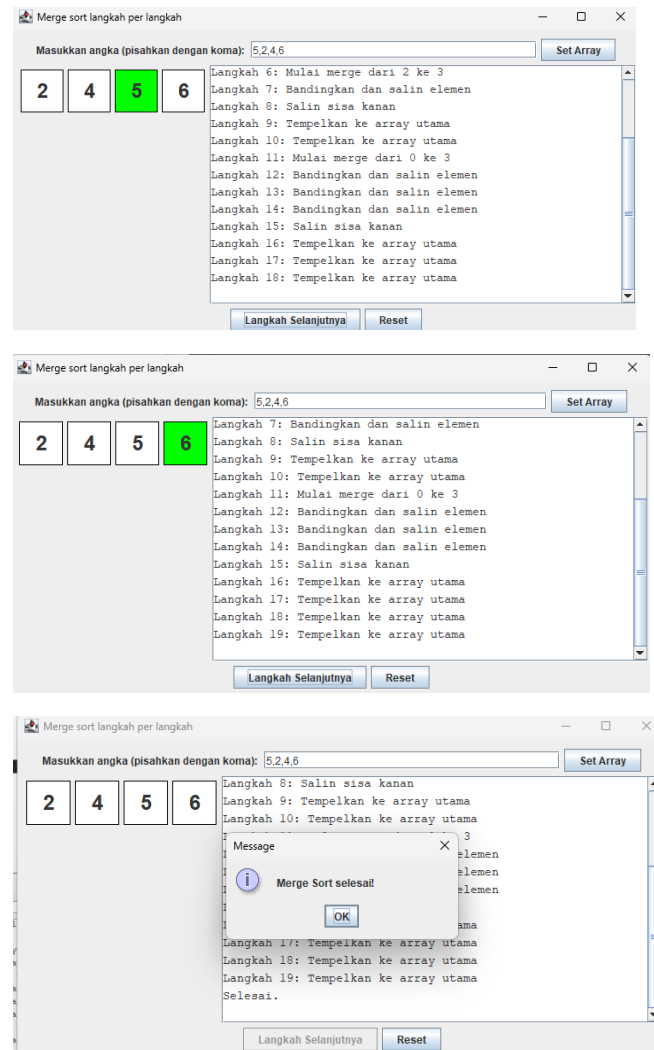
Merge sort langkah per langkah

Masukkan angka (pisahkan dengan koma): 5,2,4,6

2	5	4	6
---	---	---	---

Langkah 1: Mulai merge dari 0 ke 1
 Langkah 2: Bandingkan dan salin elemen
 Langkah 3: Salin sisa kiri
 Langkah 4: Tempelkan ke array utama
 Langkah 5: Tempelkan ke array utama
 Langkah 6: Mulai merge dari 2 ke 3
 Langkah 7: Bandingkan dan salin elemen
 Langkah 8: Salin sisa kanan
 Langkah 9: Tempelkan ke array utama
 Langkah 10: Tempelkan ke array utama
 Langkah 11: Mulai merge dari 0 ke 3
 Langkah 12: Bandingkan dan salin elemen





Gambar 2.5.3 Output kode program MergeSortGUI_2511531018.

2.6 TUGAS

2.6.1 Lagu_2511531018

```
package pekan8_2511531018;

public class Lagu_2511531018 {
    String judul_1018;
    String penyanyi_1018;
    int durasi_1018;

    Lagu_2511531018(String judul_1018, String penyanyi_1018,
int durasi_1018) {
        this.judul_1018 = judul_1018;
        this.penyanyi_1018 = penyanyi_1018;
        this.durasi_1018 = durasi_1018;
    }
}
```

Gambar 2.6.1 Kode program Lagu_2511531018.

2.6.2 Sorting_2511531018

```
package pekan8_2511531018;

public class Sorting_2511531018 {
    static Lagu_2511531018[] dataLagu_1018 = new
    Lagu_2511531018[7];

    static void inputData_1018() {

        dataLagu_1018[2] = new Lagu_2511531018("No One
        Knows", "Brent Faiyaz", 271);
        dataLagu_1018[3] = new Lagu_2511531018("You Da One",
        "Rihanna", 200);
        dataLagu_1018[0] = new Lagu_2511531018("Dreamin",
        "PARTYNEXTDOOR", 147);
        dataLagu_1018[1] = new Lagu_2511531018("bad", "wave
        to earth", 263);
        dataLagu_1018[4] = new
        Lagu_2511531018("Irreplaceable", "Beyonce", 227);
        dataLagu_1018[5] = new Lagu_2511531018("Pillowtalk",
        "Zayn", 202);
        dataLagu_1018[6] = new Lagu_2511531018("Animals",
        "Maroon 5", 231);
    }

    static void shellSort_1018() {
        int n_1018 = dataLagu_1018.length;
        for (int gap_1018 = n_1018 / 2; gap_1018 > 0;
        gap_1018 /= 2) {
            for (int i_1018 = gap_1018; i_1018 < n_1018;
            i_1018++) {
                Lagu_2511531018 temp_1018 =
                dataLagu_1018[i_1018];
                int j_1018 = i_1018;

                while (j_1018 >= gap_1018 &&
                dataLagu_1018[j_1018 -
                gap_1018]
                .judul_1018.compareToIgnoreCase(temp_1018.judul_1018) > 0)
                {
                    dataLagu_1018[j_1018] =
                    dataLagu_1018[j_1018 - gap_1018];
                    j_1018 = j_1018 - gap_1018;
                }

                dataLagu_1018[j_1018] = temp_1018;
            }
        }
    }

    static void tampilData_1018() {
```

```

        for (int i_1018 = 0; i_1018 < dataLagu_1018.length;
i_1018++) {

            System.out.println((i_1018 + 1) + ". " +
dataLagu_1018[i_1018].judul_1018 + " - " +
dataLagu_1018[i_1018].durasi_1018 + " detik");
        }
    }

    public static void main(String[] args) {
        inputData_1018();
        System.out.println("=== Sorting Playlist NIM :
2511531018 ===\n");

        System.out.println("Data Sebelum Sorting:");
        tampilData_1018();

        shellSort_1018();

        System.out.println("\nData Setelah Shell Sort (Judul
A-Z):");
        tampilData_1018();
    }
}

```

Gambar 2.6.2 Kode program Sorting_2511531018.

2.6.3 Output

```

=== Sorting Playlist NIM : 2511531018 ===

Data Sebelum Sorting:
1. Dreamin - 147 detik
2. bad - 263 detik
3. No One Knows - 271 detik
4. You Da One - 200 detik
5. Irreplaceable - 227 detik
6. Pillowtalk - 202 detik
7. Animals - 231 detik

Data Setelah Shell Sort (Judul A-Z):
1. Animals - 231 detik
2. bad - 263 detik
3. Dreamin - 147 detik
4. Irreplaceable - 227 detik
5. No One Knows - 271 detik
6. Pillowtalk - 202 detik
7. You Da One - 200 detik

```

Gambar 2.6.3 Output kode program Sorting_2511531018.

2.6.4 Penjelasan

1. Kode Program

Shell sort pada program diatas mengurutkan data berdasarkan judul, yaitu dimulai dari A-Z. Membandingkan data

yang memiliki jarak tertentu (gap), kemudian memindahkan data yang posisinya belum sesuai. Nilai gap awalnya ditentukan setengah dari jumlah data dan akan terus diperkecil hingga menjadi satu. Pada setiap proses perbandingan, judul lagu dibandingkan menggunakan `compareToIgnoreCase()` agar pengurutan tidak membedakan huruf besar dan kecil. Setelah seluruh proses selesai, data lagu akan tersusun secara alfabetis dan ditampilkan sebagai hasil pengurutan.

2. ShellSort (alasan)

Mengurutkan data dengan menggunakan ShellSort cukup mudah. Kode programnya lebih mudah untuk dipahami, mengurutkan data menggunakan jarak tertentu. Proses pengurutan juga lebih mudah dan cukup cepat dibandingkan yang lain.

BAB III PENUTUP

Algoritma sorting dapat digunakan untuk mengurutkan data mahasiswa secara alfabetis dengan lebih teratur dan mudah dibaca. Pada program ini digunakan 3 algoritma sorting, yaitu Shell Sort, Quick Sort, dan Merge Sort yang masing-masing memiliki cara kerja berbeda dalam proses pengurutan data. Selain itu, visualisasi langkah-langkah sorting membantu pengguna memahami bagaimana proses pengurutan dilakukan pada setiap algoritma. Dengan adanya program ini, pengguna dapat mempelajari konsep dasar struktur data khususnya algoritma sorting dengan lebih mudah, jelas, dan menarik.

DAFTAR PUSTAKA

- [1] R. S. Pressman, *Rekayasa Perangkat Lunak Pendekatan Praktisi*, Yogyakarta: Andi, 2015.
- [2] Rosa A.S dan M. Shalahuddin, *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek*, Bandung: Informatika, 2018.